

SESSION 2: FEDERATED LEARNING SYSTEMS

What is Federated Learning

- Decentralized training across many edge devices or clients
- Data stays on device; only model updates or gradients are shared

Why it matters

- Privacy preservation and regulatory compliance
- Lower central bandwidth use; clients compute locally

Core challenges

- Clients are intermittent and heterogeneous (CPU/GPU, battery, network)
- Non-IID data across clients; drift and imbalance
- Faults and stragglers; limited communication rounds

FedAvg (Federated Averaging) protocol

- Round t :
 - Server broadcasts current model to selected clients
 - Clients train locally for a few epochs on their data
 - Clients send model deltas or updated weights to server
 - Server aggregates (weighted average by client data size) → new global model

Practical tips

- Client sampling: pick a subset each round to scale and reduce stragglers
- Weighting: average by number of local examples to reduce bias
- Compression: quantize/sparsify updates to save bandwidth
- Security/privacy: secure aggregation, differential privacy, device attestation
- Evaluation: track both global and per-client metrics; handle skewed datasets

Client Scheduling and Participation

Why scheduling matters

- Clients are heterogeneous and intermittent; naïve selection creates bias and slow rounds

Eligibility criteria (examples)

- Sufficient battery and charging state
- Unmetered/high-quality network; device idle and not in active use
- Minimum storage/CPU/GPU/RAM; recent app/version; security/attestation passed

Selection strategies

- Random sampling from eligible pool each round
- Stratified sampling by geography, device class, or data attributes to reduce bias
- Priority/credit systems to ensure fairness and fresh participation
- Cap per-client frequency to avoid overusing fast devices

Availability patterns to consider

- Time zones and diurnal patterns
- Usage behavior (work hours vs. overnight charging)
- Network variability by region/carrier; hardware capability differences

Robustness and performance

- Over-select (invite $K \gg \text{target}$) to tolerate dropouts; accept first N that check in
- Deadlines and partial aggregation; early stopping if enough updates arrive
- Adaptive cohort size: increase when stragglers are common
- Secure eligibility checks; anonymized telemetry for scheduling
- Metrics to monitor
- Round completion time, acceptance rate, dropout rate, coverage/fairness by strata
- Contribution quality/weight (examples seen, loss improvement)

Communication- Efficient Protocols

Core idea: cut bytes per round without hurting accuracy

Techniques

- **Quantization:** send updates in 8/4/2 bits with per-tensor scales
- **Sparsification:** top-k or thresholded updates; error feedback to recover dropped mass
- **Structured updates:** layer-wise or last-layer only; low-rank adapters (LoRA/PEFT)
- **Delta/periodic sync:** send changes since last round; skip small updates

Practicalities

- Clip and add DP noise if required; secure aggregation friendly
- Calibrate k/bit-width; monitor loss drift and add error feedback
- Compress after aggregation when possible to reduce server load

SESSION 3: PRIVACY, SECURITY & DEPLOYMENT

By the end of this session, we should be able to:

- distinguish data locality, confidentiality, and formal privacy;
- explain the separate roles of differential privacy, secure aggregation, and encryption;
- identify privacy and integrity threats in federated systems;
- design deployment controls for heterogeneous, unreliable, and regulated environments.

Key idea: Federated learning keeps raw data local, but it does not automatically eliminate privacy leakage, attacks, or compliance obligations.



Training
collaboratively
doesn't
automatically
guarantee privacy

Threats

- Gradient/model inversion (reconstruct inputs)
- Membership inference (was a record in training?)
- Property inference and data poisoning/backdoors

Risk factors

- Small batches, overfitting, rich gradients, text/images

Defenses

- Secure aggregation + encryption in transit/at rest
- Differential privacy with clipping + noise
- Regularization, larger batches, early stopping; audit and red-team



Differential Privacy Mechanisms

- **Definition**
 - (ϵ, δ) -DP: output barely changes when one record is added/removed
- **Core ingredients**
 - Clip per-client update to bound sensitivity
 - Add calibrated Gaussian noise; track privacy with an accountant
- **In federated learning**
 - Do clipping + noise on device; combine with secure aggregation
 - Monitor spent ϵ, δ ; stop or adjust when budget nears limit
- **Practical tuning**
 - Set clip norm from quantiles; choose noise to hit target ϵ
 - Larger batches and fewer rounds reduce privacy cost

Secure Aggregation Protocols

Purpose: server learns only the sum of client updates—never any individual update

Approaches

- Masking with secret sharing (pairwise masks cancel in the sum; threshold recovery)
- Homomorphic encryption (server aggregates ciphertexts)

Practical notes

- Handle dropouts with threshold schemes and mask-recovery
- Combine with DP; encrypt in transit and at rest
- Validate with audits and fault-injection tests

